Boosting Semantic Segmentation of Aerial Images via Decoupled and Multilevel Compaction and Dispersion

Lianlei Shan, *Graduate Student Member, IEEE*, Weiqiang Wang, *Member, IEEE*, Ke Lv, *Senior Member, IEEE*, and Bin Luo, *Senior Member, IEEE*

***Abstract*— Semantic segmentation is a valuable task in practical applications for aerial images. Nevertheless, the segmentation performance is unsatisfactory due to aerial images' huge intraclass variance and interclass similarity. To solve this problem, we propose an approach to increase the distinction between classes and compact the features of the same class. Specifically, since a single aerial image contains only a small number of categories, which is fatal for previous contrastive learning, we discard InfoNCE loss in contrastive learning and use the simple mean square error (mse) loss that does not require negative samples to decouple the dispersion and compaction operations. Besides, we set up more representative prototypes for classes and extend the prototypes to the whole dataset level, which we call image- and dataset-level prototypes. Based on the calculated prototypes, we propose multilevel intraclass feature compaction (MFC) and multilevel interclass feature dispersion (MFD) to compact the features of the same class and disperse the features of different classes in the latent feature space. More importantly, some measures are proposed to ensure the two do not conflict. MFC and MFD can be applied to any existing segmentation network to improve performance significantly without increasing computational complexity during inference. Moreover, we feed the calculated multilevel prototypes directly into the classifier, thus keeping the feature extraction and classifier consistent. Results on four challenging datasets, Deepglobe, iSAID, Potsdam, and Vaihingen, demonstrate the significant effect of our method, and sufficient ablation studies verify the role of each module.**

***Index Terms*— Aerial image, multilevel feature interclass dispersion, multilevel feature intraclass compaction, semantic segmentation.**

I. Introduction

THE rapid development of satellite sensors enables humans to capture a large number of aerial images. Automatical analysis for these aerial images has been an open hot research issue due to its critical importance in agriculture, navigation, environmental management, land cover change detection [1], [2], [3], etc. Semantic segmentation for aerial images, also called land cover classification [4], which aims to predict a class for each pixel, is a critical basic task for the above applications.

The emergence of convolutional neural network (CNN)-based models has greatly promoted the development of semantic segmentation. The main approaches can be divided into two streams: full integration of features at different scales and attention mechanisms. For the first stream, typical works are the decoder part in U-Net [5], the pyramid pooling module (PPM) in PSPNet [6], and the atrous spatial pyramid pooling (ASPP) in Deeplab [7]. For the second stream, various attention mechanisms have achieved great success in semantic segmentation, including adding attention to each channel [8], adding attention to each pixel position [9], pyramid attention [10], and a combination of various situations [11]. Moreover, some segmentation networks, especially for aerial images, are proposed, such as GLNet [12], [13] and DENet [14]. Others also achieve great improvements in feature extraction [15], spatial relationship [11], valuable context [16], etc. However, all the above-mentioned works improve the segmentation accuracy by adding context information. And compared with natural images, aerial images have weak semantic restrictions. For example, in natural images, it is impossible for cars to run on water, for people to stand in the air, etc. On the contrary, such semantic restrictions are relatively weak in aerial images, like rivers can run through farmland, barren, and forest. Therefore, aerial image segmentation relies more on its class representation, like color and texture. Moreover, in the real world, aerial images have great intraclass variances and interclass similarities [17], [18], which is a critical bottleneck that limits the accuracy of semantic segmentation of aerial images.

Contrastive learning is an important way to solve great intraclass variances and interclass similarities. It learns representations in a differentiated way by comparing positive and negative pairs, and InfoNCE loss [19] is the core of contrastive learning. In InfoNCE loss, given an anchor prototype of a particular category, it treats prototypes of the corresponding category extracted from data augmentation as positive samples, while other prototypes from the original image as negative samples. However, this approach will lead to a

Manuscript received 1 March 2023; revised 23 May 2023 and 27 June 2023; accepted 13 July 2023. Date of publication 19 July 2023; date of current version 2 August 2023. This work was supported in part by NSFC Key Projects of International (Regional) Cooperation and Exchanges under Grant 61860206004 and in part by NSFC Projects under Grant 61976201. *(Corresponding author: Weiqiang Wang.)*

Lianlei Shan and Weiqiang Wang are with the School of Computer Science and Technology, University of Chinese Academy of Sciences, Beijing 100049, China (e-mail: shanlianlei18@mails.ucas.edu.cn; wqwang@ucas.ac.cn).

Ke Lv is with the School of Engineering Science, University of Chinese Academy of Sciences, Beijing 100049, China.

Bin Luo is with the MOE Key Laboratory of Signal Processing and Intelligent Computing, School of Computer Science and Technology, Anhui University, Hefei 230601, China.

Digital Object Identifier 10.1109/TGRS.2023.3297092

1558-0644 © 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 02:44:19 UTC from IEEE Xplore. Restrictions apply.

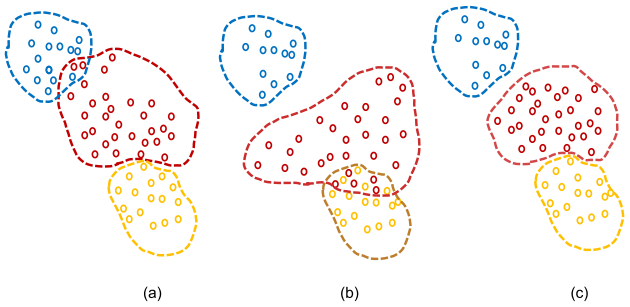


Fig. 1. Results corresponding to different operations. Each dot represents a feature. The same color represents the same category, and different colors represent different categories. Blue and red represent the two classes in the current training image, and yellow represents the other. (a) Result without any operation, and the features of each category are mixed. (b) Result after previous InfoNCE-based contrastive learning methods. The blue and red classes are indeed dispersed in the feature space, but it causes the features of the red class to invade the feature space of the yellow class, resulting in confusion between the two. Besides, the compaction of the red category is incomplete. (c) Result of our decoupled and multilevel compaction and dispersion. The features of different categories are dispersed, and each category is aggregated.

feature space structure bias, where only prototypes from partial classes (existing in the current learning images or batches) are constrained. In particular, the size of aerial images is huge and needs to be cropped into patches during training, which results in a single image containing fewer classes. Due to the feature space's limited size (the feature map's channels are restricted), anchor prototypes will be pushed to other classes in the unconstrained feature space, which may reduce the discrimination between the class of the anchor prototype and these classes in the unconstrained feature space. Unconstrained feature space denotes the feature space of classes that do not occur in the current learning image or batch. A specific example is shown in Fig. 1(b). Blue and red represent the two classes in the current training image, and yellow represents the other class. When InfoNCE loss is conducted, the blue and red classes are indeed dispersed in the feature space, but it causes the features of the red class to invade the feature space of the yellow class, resulting in confusion between the two. In a nutshell, the previous contrastive learning coupled compaction and dispersion, making it unsuitable for the characteristics of aerial images.

The other defect of the previous contrastive learning method is that the operation is only conducted on a single pixel level, resulting in the fact that the features of different categories may still relatively be mixed, i.e., dispersion and compaction are not fully performed, which makes the effects of dispersion and compaction not fully displayed. As shown in Fig. 1(b), the compaction of the red category is not complete.

To solve the above-mentioned two problems, in this article, we propose a different contrastive learning approach that decouples dispersion and compaction and directly performs multilevel operations to achieve complete compaction and dispersion. We first obtain the category of each feature based on the downsampled label and then calculate the feature center for each category in the image, which we call the image-level prototype. To avoid extreme features causing prototype shifts and weakening representation, we eliminate outlier features via the confidence of the output. Meanwhile, we also obtain and continuously update the dataset-level (global-level) prototypes based on the computed image-level prototypes. In this way, we can get multilevel representative prototypes of each class. Besides, we add a more comprehensive memory bank to store these prototypes. Based on the computed prototypes and memory bank, we decouple compaction and dispersion and conduct multilevel interclass feature dispersion (MFD) and multilevel intraclass feature compaction (MFC), respectively.

1) MFC: Given the calculated dataset-level and image-level prototypes for each class, we directly use the simple mean squared error (mse) loss to close the features to its prototype. The mse loss does not need to collect negative samples, and it only increases the similarity within the classes and does not push the features of different classes away so that the confusion problem in InfoNCE can be easily avoided. The effect of mse loss is competitive as InfoNCE when the feature is normalized because of the same optimization goal. Moreover, since multilevel prototypes can be calculated, compacting operations are performed on image level (local level) and dataset level (global level). Image-level compaction is to compact features of the same class in an image, and dataset-level compaction is to gather the image-level prototypes to the dataset-level prototypes, which allows features from the same class to be compacted in every feature space.

2) MFD: MFD is divided into pixel level, image level (local level), and dataset level (global level). As for pixel level, InfoNCE-based contrastive learning is kept as in [20], but we make some targeted improvements according to the features of aerial images. Specifically, instead of randomly selecting pixel-level features as anchors, we use the more representative prototypes as anchor features. More importantly, given the anchor feature, we only select the features closest to the anchor feature (which can be regarded as the boundary features), which reduces the calculation amount. Moreover, due to the sparse operations on dispersion, the compaction within classes will not be destroyed. The image-level MFD is to disperse image-level prototypes of different classes in an image, while the dataset level is to disperse dataset-level prototypes. Through the above operations, the effect shown in Fig. 1(c) can be achieved, i.e., dispersion and compaction can be achieved simultaneously.

MFC and MFD are carried out in the feature extraction part, and there is no corresponding operation for the classifier, which will cause inconsistency between the classifier and the feature extractor, thus resulting in errors. To solve this problem, we directly feed the prototypes at various levels into the classifier, enabling the classifier to quickly adapt to the changes in extracted features and ensure the consistency of the two.

To sum up, our main contributions are as follows.

1) According to the characteristics of aerial images and the drawbacks of InfoNCE loss, we decouple the dispersion and compaction operations and design the MFC and MFD. The proposed method can be applied to any existing segmentation network and shows remarkable results on four benchmark datasets.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 02:44:19 UTC from IEEE Xplore. Restrictions apply.

2) MFC calculates the feature center of each class and then makes each feature get close to the prototype. Besides, we use mse that does not require negative samples to protect the feature space of other classes. MFC is conducted at two levels (image level and dataset level) to ensure that features from the same class are compacted in various feature spaces.
3) MFD is on three levels: pixel level, image level, and dataset level. For pixel-level operations, to ensure the compaction of features and protect the feature space of other categories, we only calculate the difficult samples (boundary features). Image-level and dataset-level dispersion are calculated in units of prototypes, which can integrate compaction and dispersion without conflict.
4) The calculated multilevel prototypes are sent to the classifier to ensure the consistency between the feature extractor and the classifier to avoid the mismatch caused by directly changing the feature extractor without the corresponding change in the classifier.

II. Related Work

A. Semantic Segmentation

The traditional semantic segmentation methods are based on region merging, such as SLIC [21] and K-means [22], where handcrafted features are extracted. Golipour et al. [23] proposed a new spectral–spatial segmentation method, which is based on integrating hierarchical segmentation results into Markov random field (MRF) spatial prior in the Bayesian framework. With the rapid development of deep learning, many popular methods for general segmentation tasks have been proposed and achieved good results. Specifically, these studies can be broadly divided into two groups. One is to design a novel backbone network [24], [25] to extract a more robust feature representation for each pixel. The other approach is to introduce richer contextual information for each pixel, including multiscale feature fusion and attention mechanisms. The atrous convolution and atrous spatial pyramid pool modules in Deeplab [7] extended the receptive field and thus created connections between pixels that are far apart. PSPNet [6] combined with feature maps of different scales to enlarge the receptive field and achieved the same effect as Deeplab. Attention mechanisms have achieved great success in semantic segmentation, including spatial attention and channel attention [8], and a combination of the two [9]. LANet [26] proposed a patch attention module (PAM) and attention embedding module (AEM) to enhance features with high-level semantic information and low-level detail information. Other works [27] also use various levels of attention and achieve great success, such as edge attention and relation attention. The above analysis shows that the motivation of the above-mentioned works is to increase accuracy by expanding context information.

However, in the semantic segmentation of aerial images, especially in the rural land cover classification, the spatial dependencies of classes are weak, and the network is more dependent on the features of pixels for prediction. We find the characteristics of aerial images and propose an improvement measure based on the organization of feature space.

B. Contrastive Learning

Contrastive learning learns representations in a differentiated way by comparing positive and negative pairs. More recently, several methods based on contrastive loss [28] have been proposed for self-supervised visual representation learning [29], [30]. These approaches treat each instance as a class and use instance differentiation based on contrastive losses for presentation learning. Specifically, they use an expanded version of one instance to form positive pairs and other randomly sampled instances to form negative pairs to compensate for the loss of contrast.

Some works proposed a fully supervised pixel-pixel contrastive learning semantic segmentation method and a new training protocol that explores global pixel relationships for regularizing embedded space. Contrastseg [20] is a landmark effort. It computes the class relationship between different images so that contrastive learning is not limited to a single image. It also allows for comparison at different levels, not just between pixels. But the above work uses InfoNCE loss [19], so a batch needs to contain as many classes as possible (preferably containing all classes), which is very difficult for aerial images. Due to the large size of aerial images, images need to be cropped during training, which further exacerbates the phenomenon that an image contains few classes.

In our work, compaction and dispersion are decoupled, and specific losses are designed for the two operations, respectively, which eliminates the need to require the number of categories in an image. Moreover, the proposed operations are conducted on multilevel feature spaces to make dispersion and compaction through.

C. Memory Bank Module

The feature memory module enables the deep neural network to capture valid information beyond the current images. It has shown great ability in various computer vision tasks [31], [32], [33]. For example, MEGA [31] adopted the memory mechanism and added a candidate frame's feature for detecting video objects' keyframes. For image search and a small number of shot learning tasks, MNE [32] proposed to enhance the general convolution features using neighborhood embedding based on memory. In deep metric learning, XBM [33] designed a cross-batch storage module for mining information samples across multiple small batches.

Contrastseg [20] has designed a simple and effective feature memory module for storing representations of various classes, enabling the network to capture contextual information beyond the input images. We extend the memory module to retain and update the class prototype calculated from the entire training set level.

III. Method

A. Overview

The overall procedure is shown in Fig. 2. The position of MFD and MFC is after feature extraction or aggregation and before the classifier. For example, the position is after ASPP for Deeplabv3+ [34] and after PPM for PSPNet [6].

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 02:44:19 UTC from IEEE Xplore. Restrictions apply.

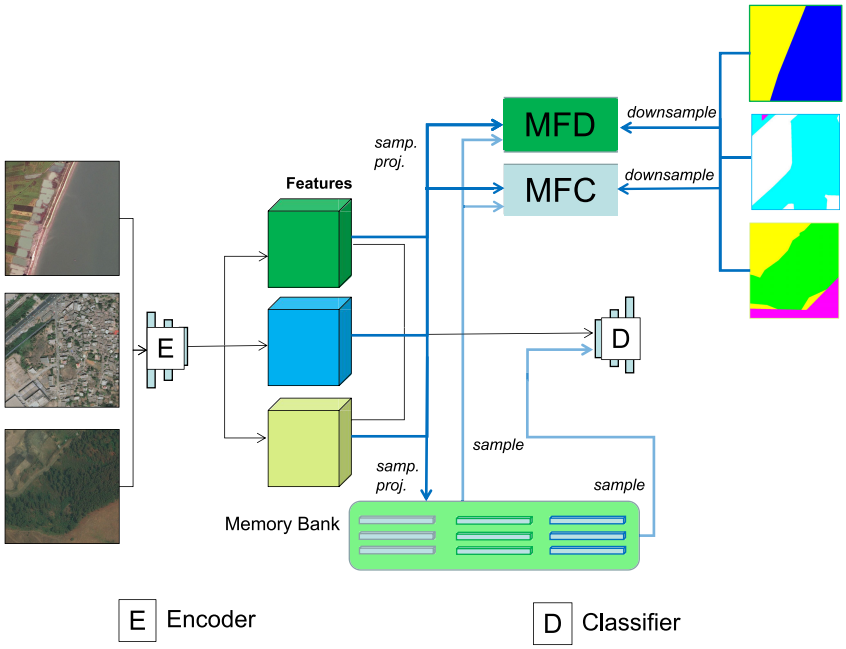


Fig. 2. Overview of the overall process. The black lines represent the process of previously common segmentation networks. *pro.* denotes project and *samp.* denotes sample. We use batch size 3 in the figure for the introduction, but other values are also acceptable. The modules we added are marked in blue lines, and they are multilevel dispersion, multilevel compaction, and memory bank. The projection is one convolution layer to reduce feature dimension. The convolution layer before the classifier contains the same structure as the projection module but does not share parameters. The encoder takes in three input images and outputs the features of the three images. The size of the feature varies according to the backbone. The shape information of various features and prototypes of the memory bank will be introduced in detail in Fig. 3. The line from feature to memory bank refers to using the feature to update the memory bank. The line from feature to MFC and MFD is to select features to participate in the compaction and dispersion operations.

As shown in Fig. 2, the input of MFC and MFD consists of three parts, i.e., the extracted features, downsampled labels with the same size as the extracted feature, and features sampled from the memory bank. The corresponding class of each feature can be obtained through the downsampled labels. We selected samples from the extracted features as anchors (the feature to be compared with others) and then selected some features from the memory bank and extracted features for comparison according to the selected anchors. MFC and MFD are conducted based on the calculated prototypes and selected features. Besides, there are also some features to update the memory bank.

The memory bank consists of three parts, pixel-level features, local prototypes, and global prototypes. Pixel features are sampled from the extracted features after a projection. The projection is one 1×1 convolution layer to reduce channel dimension and the memory bank space. The local prototype is the average feature of the same class in an image. Unlike the previous [20], we improve the calculation method of the local prototype, eliminating elements with low confidence, thus making the prototype more representative. The global prototype is the average of the features for a class in the entire training set, which is calculated and continually updated from local prototypes.

We introduce the memory bank in Section III-B, including the calculation of the local prototype and global prototype. MFC and MFD are introduced in Section III-C and III-D. In Section III-E, we introduce the operations of the corresponding adjustment of the classifier. The overall loss is introduced in Section III-F.

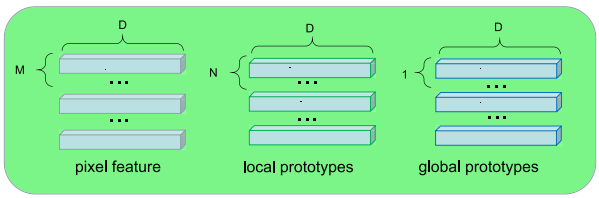


Fig. 3. Components and their sizes of the memory bank. There are M pixel features for each class, N local prototypes for each class, and one global prototype for each class. All the features are in D dimensions.

B. Memory Bank and Representative Prototypes

The memory bank consists of three parts: pixel-level features, local prototypes, and global prototypes, as shown in Fig. 3. We use C to denote the number of classes. And there are M pixel-level features for each class, so there are a total of $C \times M$ pixel-level features, and each feature is $1 \times D$. Similarly, for local prototypes, there are N samples in each class. Each class has a global prototype. All the features are D-dimension. So, the size of the memory bank is $C \times (M + N + 1) \times D$. Pixel-level features are randomly sampled from image features, and local prototypes are selected from the calculated mean value of features. Note that all the features and prototypes are normalized after projection. We improve the calculation of prototypes to make them more representative and propose a global prototype, which is introduced next.

1) Representative Local Prototypes: The previous methods [20] directly calculate the mean of features as the prototype. Formally, we use $\mathbf{F}$ to denote the extracted features

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 02:44:19 UTC from IEEE Xplore. Restrictions apply.

of an image, and $\mathbf{f}_i$ denotes the feature of the pixel i from $\mathbf{F}$. $\mathbf{y}$ is the label for feature $\mathbf{F}$, and y_i is the corresponding label for $\mathbf{f}_i$. Therefore, the prototype $\hat{\mathbf{p}}_c$ for class c in previous works is

$$\hat{\mathbf{p}}_c = \frac{\sum_{\mathbf{f}_i \in \mathbf{F}} \mathbf{f}_i \mathbb{1}\left[y_i = c\right]}{\left|\mathbb{1}\left[\mathbf{y} = c\right]\right|} \tag{1}$$

where $\mathbb{1}[\mathbf{y} = c]$ indicates that when the label corresponding to the feature is class c, the value is 1; otherwise, the value is 0. $|\ \mathbb{1}[\mathbf{y} = c]|$ denotes the total number of features whose class is c.

However, this approach is not suitable for aerial images. When calculating prototypes, it is only based on ground truth (label) and regardless of the confidence of the output. The consequence is that the wrong predicted features (outliers) are also involved in prototype calculations, making the prototype less typical. To solve this problem, we bring the output of the network to filter the reliable features. Simply put, we do not consider the features that are not well extracted (outlier features) but only select features with high output confidence. We use $\mathbf{y}^*$ to denote the prediction score (after softmax) of the classes, which corresponds to $\mathbf{y}$ for feature $\mathbf{F}$. Our prototype $\hat{\mathbf{p}}_c$ for class c is

$$\hat{\mathbf{p}}_c = \frac{\sum_{\mathbf{f}_i \in \mathbf{F}} \mathbf{f}_i \mathbb{1}\left[y_i = c \cap y_i^* > \beta\right]}{\left|\mathbb{1}\left[\mathbf{y} = c \cap \mathbf{y}^* > \beta\right]\right|} \tag{2}$$

where β is the threshold of scores. We sort all the elements according to scores from large to small and then select the first $K\%$, and β is the score of the borderline features at $K\%$.

2) Global Prototypes: On the other hand, we design the global prototype to calculate prototypes from the whole dataset level. The global prototype is gradually updated by local prototypes. We use t to denote the number of images containing class c passed by the network, so t can be regarded as steps. $\mathbf{p}_c[t-1]$ denotes the global prototype of class c at step $t-1$. The calculation of the global prototype at step t is

$$\mathbf{p}_c[t] = \frac{1}{t}\left((t-1)\mathbf{p}_c[t-1] + \hat{\mathbf{p}}_c\right). \tag{3}$$

In short, the latest global prototype is the weighted average of the previous global prototype and the current local prototype. All the global prototypes refer in this article are the latest prototypes, so we omit the step t for brevity. Global prototypes can be used for intraclass feature compaction and interclass feature dispersion from the whole dataset level.

C. Multilevel Intraclass Feature Compaction

This section shows how to use the computed prototypes to perform MFC. Specifically, there are two types of MFC, local level and global level, which are, respectively, introduced in the following parts.

1) Local-Level Compaction: Fig. 4 shows the procedure of local-level intraclass feature compaction. The whole process can be split into three parts: get the label, select feature, and compaction. To obtain the label, the ground truth (original size label) is downsampled to the same size as the feature map in the spatial dimension, and then each feature can get the class by the corresponding position label. Then, we compute

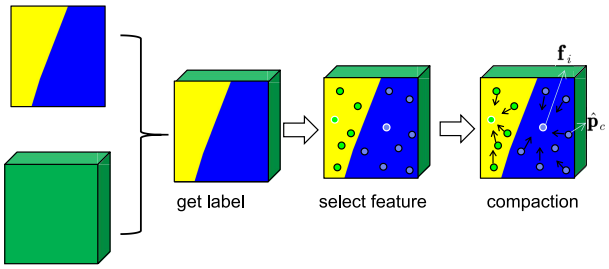


Fig. 4. Overview of the local-level compaction. The colors are consistent with Fig. 2. The whole process is divided into three steps: getting labels, selecting features, and compaction. Each dot represents a feature, and the dots with white borders represent the calculated local prototypes. Our goal is to get features of the same class closer to its prototype.

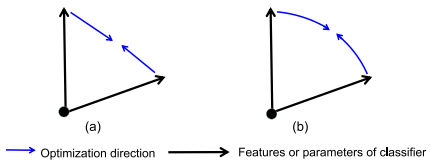


Fig. 5. (a) Optimization of mse loss, and the optimized goal is to shorten the square distance of the two extracted features. (b) Optimized goal that brings the inner product of the two features smaller. After normalization, the functions of the two are similar, i.e., network optimization using mse loss can also lessen the inner product of features.

the local prototype according to (2) and randomly select the features for comparison. Finally, the intraclass compaction is performed on these features. Compaction is the core operation, and we will introduce it in detail next.

Formally, we use $\hat{\mathbf{p}}_c$ to denote the local prototype of class c of an image, and $\mathbf{f}_i$ denotes a feature sampled from extracted features. y_i is the corresponding label of $\mathbf{f}_i$. $\mathcal{C}$ denotes classes of this image. And the calculation of local prototype level compacting loss $\mathcal{L}_{\text{cpt}}^{l}$ (l denotes local prototypes and cpt denotes compaction) is

$$\mathcal{L}_{\text{cpt}}^{l} = \frac{1}{N_c} \sum_{c \in \mathcal{C}} \sum_{\mathbf{f}_i \in \mathbf{F}} \left\| \left(\mathbf{f}_i - \hat{\mathbf{p}}_c\right) \mathbb{1}\left[y_i = c\right] \right\|_2 \tag{4}$$

where N_c is the number of classes. $\mathbb{1}[y_i = c]$ denotes that the value is 1 when y_i is class c; others is 0. The location of $\mathbf{f}_i$ and $\hat{\mathbf{p}}_c$ is shown in Fig. 4.

Furthermore, we discard the original InfoNCE loss [19] and use a more direct mse loss to easily achieve the two new level compaction. If the features are normalized, mse loss is equivalent to the cosine distance or inner product-based losses, as shown in Fig. 5. And compared with InfoNCE, mse does not need to select negative samples when calculating, so it is robust and straightforward. Specifically, the size of aerial images is very large and needs to be cropped into patches during training, which results in a single image containing fewer classes. And due to the feature space's limited size (the feature map's channels are restricted), anchor prototypes will be pushed to the unconstrained feature space, which may reduce the discrimination between the class of the anchor prototype and these classes in the unconstrained feature space. However, mse loss in MFC only increases the similarity within the class and does not push the features of different classes away, so this problem can be easily avoided.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 02:44:19 UTC from IEEE Xplore. Restrictions apply.

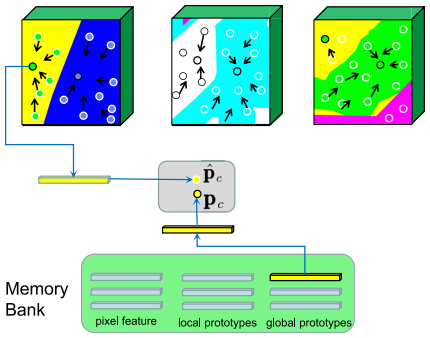


Fig. 6. Overview of global-level compaction. Each dot represents a feature, and the dots with black borders represent the calculated local prototypes. $\hat{\mathbf{p}}_c$ is the image-level (local-level) prototype, and $\mathbf{p}_c$ is the dataset-level (global-level) prototype. The goal of global-level compaction is to make local-level features of the same class as consistent as possible with the global-level prototype.

2) Global-Level Compaction: The entire global-level compaction is shown in Fig. 6. The global-level prototype is taken from the memory bank, and the local-level prototypes are calculated from the images. Formally, the global prototype-level compacting loss $\mathcal{L}_{\text{cpt}}^{g}$ is

$$\mathcal{L}_{\text{cpt}}^{g} = \frac{1}{N_c} \sum_{c \in \mathcal{C}} \left\| \left(\hat{\mathbf{p}}_c - \mathbf{p}_c \right) \right\|_2 \tag{5}$$

where $\mathbf{p}_c$ is the global prototype of class c, and $\hat{\mathbf{p}}_c$ is the local prototype. The positions of $\hat{\mathbf{p}}_c$ and $\mathbf{p}_c$ are shown in Fig. 6. The optimization goal is to make the global and local prototypes for each class more consistent.

The local-level compaction makes the features converge to the image-level prototype, and the global-level compaction makes the image-level prototype converge to the dataset-level prototype. In this way, the features of the same category can be clustered in the feature space at the entire dataset level.

D. Multilevel Interclass Feature Dispersion

MFD can be divided into three levels: pixel-level dispersion, local-level dispersion, and global-level dispersion, which will be introduced next.

1) Pixel-Level Dispersion: Pixel-level dispersion is to perform dispersion operation on pixel-level features. For pixel-level dispersion, we need to select the most representative feature as an anchor first. As discussed in the Introduction, the key to compaction operations is not infringing the feature space of other classes and ensuring that compactions within classes are not destroyed. Therefore, for the features for comparison, we cherry-pick only the features most easily confused with anchor features. In this way, features are dispersed in confusing parts, but intra-class compaction and positional relationships with other classes are not affected. Based on the above analysis, the overall sampling process is divided into two steps.

1) *Step 1:* For each class, the local prototype with a good representation is selected as the anchor feature.
2) *Step 2:* For each anchor feature, the similarity between the features of other classes and the anchor feature is sorted from high to low, and only V features with the highest similarities are selected for calculation.

V is a hyperparameter, and the similarity is calculated by the inner product.

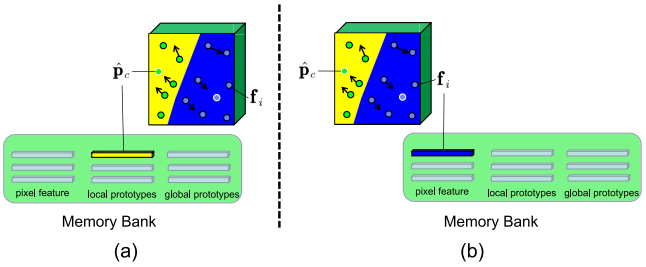


Fig. 7. Overview of interclass dispersion. Each dot represents a feature, and the dots with white borders represent the calculated local prototypes. (a) $\mathbf{f}_i$ is used as anchor feature, and $\hat{\mathbf{p}}_c$ comes from the extracted features and the memory bank. (b) $\hat{\mathbf{p}}_c$ is used as anchor feature, and $\mathbf{f}_i$ comes from the memory bank at this time. The operation to get the label is the same as in Fig. 4, so we just show the dispersion part. The goal of dispersion is to get features from different classes far from each other.

Formally, for pixels belonging to different classes, we should make their corresponding features as different as possible. So the pixel-level dispersing loss $\mathcal{L}_{\text{dps}}^{p}$ (p denotes pixel level and dps denotes dispersion) is

$$\mathcal{L}_{\text{dps}}^{p} = \frac{1}{N_c} \sum_{c \in \mathcal{C}} \sum_{\mathbf{f}_i \in \mathbf{F}} \frac{\mathbb{1}\left[y_i \neq c \right]}{\left\| \left(\mathbf{f}_i - \hat{\mathbf{p}}_c \right) \right\|_2}. \tag{6}$$

Fig. 7 shows the feature sources used for computation. There are two situations of pixel-level dispersion according to the source of the anchor feature, as shown in Figs. 7(a) and (b) of Fig. 7. Fig. 7(a) represents that $\mathbf{f}_i$ is used as the anchor feature, i.e., $\mathbf{f}_i$ is determined first, and then $\hat{\mathbf{p}}_c$ is selected according to $\mathbf{f}_i$. In this case, $\hat{\mathbf{p}}_c$ comes from two parts: extracted feature and memory bank. The reason for choosing the prototype from the memory bank is: in aerial images, an image or a batch of images often contains a small number of categories. By storing the prototype in the memory bank, the features can be compared with the prototype of every category, thus improving the effect. Fig. 7(b) indicates that $\hat{\mathbf{p}}_c$ is used as the anchor feature, and $\mathbf{f}_i$ only comes from the memory bank. In Fig. 7(b), the reason why $\mathbf{f}_i$ is not selected from the extracted features is that the case where both $\mathbf{f}_i$ and $\hat{\mathbf{p}}_c$ come from the extracted features have already been computed in Fig. 7(a).

2) Global-Level and Local-Level Dispersion: Local-level dispersion is to disperse local prototypes of different classes, while global-level dispersion is to disperse global prototypes. Both the operations are prototype-based, so they do not break the compaction within the class. Moreover, these two levels are to disperse from a higher level, thus making the dispersion more complete.

For local-level dispersing loss $\mathcal{L}_{\text{dps}}^{l}$, its calculation is

$$\mathcal{L}_{\text{dps}}^{l} = \frac{1}{N_c} \sum_{c_i \in \mathcal{C}} \sum_{c_j \in \mathcal{C} \setminus c_i} \frac{1}{\left\| \left(\hat{\mathbf{p}}_{c_i} - \hat{\mathbf{p}}_{c_j} \right) \right\|_2} \tag{7}$$

where $c_j \in \mathcal{C} \setminus c_i$ denotes that c_i and c_j are different classes. Among them, the anchor feature ($\hat{\mathbf{p}}_{c_i}$) comes from

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 02:44:19 UTC from IEEE Xplore. Restrictions apply.

the extracted feature, and the feature for comparison ($\hat{\mathbf{p}}_{c_j}$) comes from the extracted feature and the memory bank, which is to make each feature compared with features of all other categories. And for global prototype-level dispersing loss $\mathcal{L}^l_{\text{dps}}$, we only change the local prototypes to global prototypes, and the calculation is

$$\mathcal{L}^g_{\text{dps}} = \frac{1}{N_c} \sum_{c_i \in \mathcal{C}} \sum_{c_j \in \mathcal{C} \setminus c_i} \frac{1}{\left\| \left(\mathbf{p}_{c_i} - \mathbf{p}_{c_j} \right) \right\|_2}. \tag{8}$$

Unlike pixel level, all the prototypes participate in the loss rather than needing sampling. Global-level and local-level operations make prototypes dispersed, and compaction is also implemented through prototypes, so these two operations conduct dispersion while ensuring that the compaction is not destroyed.

E. Consistency of Classifier and Feature Extractor

Feature extractor and classifier should be consistent, i.e., the extracted features should be put into the matching classifier to obtain correct results. However, all the above operations are taken in the feature extraction part, such as dispersion and compaction. There is no corresponding operation for the classifier, which leads to a certain inconsistency between the feature extractor and the classifier. It is known that the parameters of the classifier can be considered as the prototype of classes [35] so that the inner product of the classifier parameters and samples (features of pixels) after normalization can represent the probability that the sample belongs to which class. Based on the above analysis, we put the prototypes stored in the memory bank into the final classifier (a 1×1 convolution) to calculate the cross-entropy loss so that the parameters of the classifier are closer to the center of the category more directly. Note that all the features in the memory bank, including the selected pixel features, local prototypes, and global prototypes, are involved to fully ensure the consistency of the classifier and prototype.

We use $\mathcal{L}^m_{\text{ce}}$ to denote the cross-entropy loss of prototypes from the memory bank, and $\mathcal{L}^f_{\text{ce}}$ for currently extracted features, and the whole optimization goal is introduced in the following parts.

F. Optimization Goal

Based on the original cross-entropy loss, our work increases the loss of multilevel interclass dispersing losses, intraclass compacting losses, and the loss of maintaining the consistency of the feature extractor and classifier. Therefore, the overall optimization goal of the network is shown as

$$\mathcal{L} = \mathcal{L}^f_{\text{ce}} + \mathcal{L}^m_{\text{ce}} + \lambda_1 \left(\mathcal{L}^l_{\text{cpt}} + \mathcal{L}^g_{\text{cpt}} \right) + \mathcal{L}^p_{\text{dps}} + \lambda_2 \left(\mathcal{L}^l_{\text{dps}} + \mathcal{L}^g_{\text{dps}} \right) \tag{9}$$

where λ_1 and λ_2 are the coefficient of losses. $\mathcal{L}^f_{\text{ce}}$ is the cross-entropy of extracted features as usually used in segmentation. $\mathcal{L}^m_{\text{ce}}$ is also cross-entropy loss, but the input for the classifier is the prototypes from the memory bank. We do experiments to verify the effect of different coefficients.

IV. Experiments

To verify the effectiveness of the proposed method, we do sufficient experiments. In Section IV-A, we introduce the implementation details, including the introduction of the used datasets, the implementation details, and the evaluation metric. In Section IV-B, we present the experimental results on Deepglobe and iSAID. In Section IV-C, we conduct an in-depth analysis of the reasons for the improvement of our method in terms of feature responses. We conduct experiments with different segmentation networks to show the generalizability of our proposed method in Section IV-D. In Section IV-E, we show the results on Potsdam and Vaihingen and perform the analysis of the limitation and future works. We make a comparison with the plug-in contrastive learning methods in Section IV-F. In Section IV-G, we analyze the computational complexity. Finally, we do ablation studies in Section IV-H, including respective roles of compaction and dispersion, results with different level operations, results with different memory bank sizes, results without consistency of feature extractor and classifier, results with different coefficients, and results with different thresholds of β.

A. Implementation Details

1) Datasets: Deepglobe contains 803 ultrahigh-resolution images (2448×2448 pixels). We split images into training, validation, and testing sets with 455, 207, and 142 images, respectively, as in [12]. The dense annotation contains seven classes of landscape regions, where one class out of seven, called the "unknown" region, is not considered in the challenge. The other six classes are urban, agriculture, rangeland, forest, water, and barren.

iSAID [17] dataset consists of 2806 HSR remote sensing images. These images were collected from multiple sensors and platforms with multiple resolutions. The original image sizes range from $\sim 800 \times 800$ pixels to $\sim 4000 \times 13000$ pixels. The iSAID dataset provides 6 55 451 instances of annotations over 15 categories of the object, which is the largest dataset for instance segmentation in the HSR remote sensing imagery. The predefined training set contains 1411 images, while the validation (val) set contains 458 images, and the test set has 937 images. In this work, we only use semantic mask annotations for object segmentation. And we use the predefined training set to train models and evaluate on the validation set because the test set is unavailable.

Vaihingen dataset is composed of 33 aerial images collected of the city Vaihingen. The average size of each image is 2494×2064 pixels. Among these images, 16 of them are manually annotated with pixelwise labels, and each pixel is classified into one of six land cover classes (buildings, cars, low vegetation, impervious surfaces, trees, and clutter/background). Following the setup in [11], we select 11 images for training, and the remaining five images (image IDs: 11, 15, 28, 30, and 34) are used to test our model.

Potsdam dataset consists of 38 high-resolution aerial images. The size of all the images is 6000×6000 pixels, which are annotated with pixel-level labels of six classes as the Vaihingen dataset. To train and evaluate networks, we use

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 02:44:19 UTC from IEEE Xplore. Restrictions apply.

10 images for training and build the test set with the remaining images (image IDs: 02 11, 02 12, 04 10, 05 11, 06 07, 07 08, and 07 10), which follows the setup in [11].

2) Implementation Details: In our experiments, various backbones and segmentation networks (i.e., DeepLabV3+ [34] and PSPNet [6]) are used to thoroughly verify the proposed methods. To ensure fairness, we initialize all the backbones using the corresponding weights pretrained on ImageNet [36], and the rest of the layers are randomly initialized. For data augmentation, we use color jitter, horizontal flipping, and random scaling with a factor of [0.5, 2]. We used SGD as our optimizer with a momentum of 0.9 and a weight decay of 0.0005. We use a polynomial decay policy [7] to schedule the learning rate, which is multiplied by $(1 - (\text{iter})/(\text{total_iter}))^{\text{power}}$ with power $= 0.9$. For pixel-level compaction, 50 anchor features are sampled for each class. For local prototypes as the anchor, all the extracted features participate in the loss calculation. For global prototypes as anchors, all the local prototypes calculated from extracted features participate in the loss. For pixel-level dispersion, 1024 features are sampled for each class, i.e., V in Section III-D is 1024. For local and global levels, as same with compaction, all the prototypes are sampled. We chose a mini-batch size of 16 with an initial learning rate of 0.001. We use both sequential cropping and random cropping. Unless otherwise specified, 200-pixel features and local prototypes for each class are stored in the memory bank. The projection dimension of the feature is set to 720. When calculating local prototypes, we select the features in the top 90% confidence to calculate the mean value. The coefficients λ_1 and λ_2 in the losses are all set to 10 unless otherwise specified in the ablation study. Note that no changes or additional inference steps are introduced to the basic segmentation model during the test. Namely, the projection head and the memory bank $\mathbf{M}$ are directly discarded in testing.

For the iSAID dataset [17], to ensure fairness, we use the same training strategies as previous works [47]. The training and validation samples are cropped with an 896×896 sliding window from the training and testing images. A stride of 512 is used for sampling the training samples, while no stride is used for sampling the testing samples. The training samples are augmented with random scaling and flipping. The networks are trained with an SGD optimizer, and batch size $= 8$. An initial learning rate of 0.007 with a polynomial decay policy is used. All the momentum is set to 0.9, and the initial weight decay is set to 0.0001. All the backbones are pretrained on the ImageNet dataset [36] due to the performance contribution of an initialization.

3) Evaluation Metrics: We use mean intersection over union (mIoU), the overall accuracy (OA), and mean F1 as the evaluation metrics. mIoU is the ratio of the intersection and union of two sets of true and predicted values and can be evaluated by

$$\text{mIoU} = \frac{1}{K}\sum_{i=1}^{K}\frac{p_{ii}}{\sum_{j=1}^{K} p_{ij} + \sum_{j=1}^{K} p_{ji} - p_{ii}} \tag{10}$$

where K is the number of classes, i is the true class label, and j is the predicted label. p_{ij} denotes the number of pixels of class i predicted to class j.

mean F1 score is defined as

$$\text{mean F1} = \left(1 + \beta^2\right)\frac{\text{precision} \cdot \text{recall}}{\beta^2\text{precision} + \text{recall}} \tag{11}$$

where β is the equivalent factor between precision and recall. β is set as 1.

B. Results on Deepglobe and iSAID

To fully and comprehensively show the effect of the method, we conduct experiments on two large and challenging datasets, iSAID and Deepglobe.

1) Deepglobe: The experimental results on Deepglobe are shown in Table I. It can be seen that the proposed method achieves a significant improvement over the previous state-of-the-art (SOTA) methods. Compared with the baseline Deeplabv3+ [34], we obtain a 4.5% gain with ResNet-50 and a 6.2% gain with ResNet-101. Contrastseg [20] performs worse than Deeplabv3+ with ResNet-50 as the backbone. Although there is little improvement when ResNet-101 is used as the backbone, the improvement is not obvious. Contrastseg is better than Deeplabv3+ when using ResNet-101, but Deeplabv3+ is better when using ResNet-50, which can be explained by the size of the feature space and the characteristics of aerial images. Aerial images contain few categories in an image or batch, so using the original InfoNCE-based contrastive learning squeezes the unconstrained space, which causes Contrastseg to work poorly compared with Deeplabv3+. When using ResNet-101 as the backbone, the feature space is greatly increased, which makes the categories in the unconstrained space less squeezed, so Contrastseg works better. Compared with the impressive improvement of contrastive learning in the domain of natural images, the effect of contrastive segmentation on aerial images is not significant, which is explainable. The large class number in a batch has a great impact on contrastive segmentation, and aerial images cannot meet this requirement. Our method separates the operations of intraclass compaction and interclass dispersion. It does not require the number of classes in the batch, which is very suitable for the characteristics of aerial images, so our method achieves very good results.

In the Deepglobe dataset, we do experiments based on different backbones. It can be seen that the results of ResNet-101 are significantly better than that of ResNet-50. The phenomenon is also easy to explain. ResNet-101 contains a larger feature space (more channels on feature maps), so the possibility of overlapping and conflicting regions for different classes in the feature space is greatly reduced.

2) iSAID: The experimental results on iSAID are shown in Table II. Compared with the baseline Deeplabv3+ [34], we obtain a 5.9% gain. Meanwhile, our method achieves a 3.0% improvement compared with the SOTA method BSNet [47]. On the iSAID dataset, BSNet achieves a very significant improvement using dynamic hybrid gradient convolution. And our method outperforms BSNet, which illustrates the strength of our method, i.e., our method does not change the network structure, and the improvement of segmentation performance is greater. Compared with some attention-based

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 02:44:19 UTC from IEEE Xplore. Restrictions apply.

TABLE I
RESULTS ON DEEPGLOBE

Method	Backbone	Urban	Agriculture	Rangeland	Forest	Water	Barren	mIoU (%)
ICNet [37]	ResNet-50	24.3	69.2	31.3	24.2	42.1	35.2	37.7
FCN	ResNet-50	26.2	72.3	36.4	35.5	38.9	36.1	40.9
U-Net [5]	ResNet-50	26.1	71.7	33.5	28.2	45.1	34.1	39.8
PSPNet [6]	ResNet-50	48.2	74.7	36.9	53.6	64.2	39.3	52.8
SegNet [38]	ResNet-50	53.1	75.2	38.2	54.8	67.8	43.1	55.4
Contrastseg [20]	ResNet-50	65.6	79.3	36.2	68.8	69.9	55.4	62.5
DeepLabV3+ (baseline) [34]	ResNet-50	62.2	78.9	38.7	70.2	70.4	58.5	63.2
Ours	ResNet-50	**72.6**	**82.2**	**41.6**	**72.4**	**75.2**	**62.2**	**67.7**
PSPNet [6]	ResNet-101	49.1	75.4	37.4	55.8	65.2	45.0	54.7
SegNet [38]	ResNet-101	67.9	80.7	41.6	73.4	75.4	62.3	66.9
Contrastseg [20]	ResNet-101	74.2	83.0	43.7	75.1	78.1	64.1	69.7
HRNet+OCR [39]	HRNet-W48	72.4	84.1	40.9	74.0	79.9	65.7	69.5
DeepLabV3+ (baseline) [34]	ResNet-101	70.2	81.4	42.7	74.2	76.1	63.4	68.0
Ours	ResNet-101	**79.7**	**88.9**	**45.4**	**77.8**	**86.6**	**66.8**	**74.2**

TABLE II
RESULTS ON iSAID

Method	Backbone	PL	BD	BR	GTF	SV	LV	SH	TC	BC	ST	SBF	RA	HA	SP	HC	mIoU(%)	mean F_1	OA
EMANet [40]	ResNet-50	77.8	71.2	28.3	53.1	40.0	56.9	58.4	82.6	52.4	62.8	63.5	40.2	48.7	43.1	30.8	56.8	70.3	94.0
DenseASPP [41]	ResNet-50	78.2	66.9	28.4	53.5	38.2	57.4	60.8	82.4	53.7	50.1	69.9	54.8	51.2	43.4	25.7	57.1	70.6	94.2
Non-local [42]	ResNet-50	81.0	71.9	29.1	49.8	47.8	59.1	62.4	84.9	48.6	62.2	41.6	53.5	50.9	45.1	29.9	57.3	70.8	94.3
DANet [9]	ResNet-50	80.9	71.4	28.6	52.5	42.1	57.5	60.2	84.7	50.9	63.0	63.3	40.6	48.8	46.1	30.4	57.5	71.0	94.4
CCNet [43]	ResNet-50	78.8	72.4	30.5	52.2	44.4	53.1	56.3	85.3	49.2	63.2	67.1	51.8	48.7	43.6	29.2	57.8	71.3	94.6
Semantic-FPN [44]	ResNet-50	81.2	71.5	33.8	52.2	45.4	60.1	63.5	87.1	57.8	61.5	60.2	59.0	51.5	46.6	31.2	60.1	73.5	94.8
SETR [45]	T-Large	81.5	74.5	30.8	52.3	**48.8**	58.6	63.4	86.9	57.4	67.9	68.0	61.6	52.9	46.2	32.6	61.4	74.4	95.3
UperNet [46]	Swin-B	**83.0**	75.3	30.5	51.8	49.6	59.2	63.1	85.7	59.2	68.2	68.1	59.5	53.3	46.8	34.3	61.6	74.7	95.5
PSPNet [6]	ResNet-50	82.8	76.8	31.2	53.4	47.3	58.2	63.9	86.5	56.7	68.3	67.4	52.6	53.0	44.0	33.3	60.9	74.2	95.1
BSNet [47]	ResNet-50	81.8	76.1	41.3	50.3	46.6	**63.4**	65.3	86.8	55.9	69.1	70.2	70.0	**57.3**	48.8	31.8	63.4	76.6	96.4
DeepLabV3+ (baseline) [34]	ResNet-50	82.9	71.4	32.2	52.0	48.5	60.1	64.5	81.1	55.7	68.5	61.0	60.2	53.0	47.1	30.8	60.5	73.9	94.8
Ours	ResNet-50	80.7	**77.0**	**42.0**	**61.3**	46.4	63.0	**67.0**	**89.1**	**63.3**	**72.2**	**80.0**	**71.9**	57.2	**52.2**	**41.1**	**66.4**	**78.4**	**97.0**

iSAID includes 15 foreground categories, i.e., plane (PL), baseball diamond (BD), bridge (BR), ground track field (GTF), small vehicle (SV), large vehicle (LV), ship (SH), tennis court (TC), baseball court (BC), storage tank (ST), soccer ball field (SBF), roundabout (RA), harbor (HA), swimming pool (SP) and helicopter (HC)

methods such as nonlocal, DANet, and CCNet, our method also shows better performance.

3) Visualization Results: The visualization results are shown in Fig. 8. Our method has achieved great improvements in the easily confused boundary regions, such as water and rangeland in the first row and forest and rangeland in the second row. For some easily confused categories, such as barren in the fourth row, FCN classifies it as rangeland, Deeplabv3+ classifies it as water, and Contrastseg classifies it partially correctly but not completely. And our method makes a good distinction between rangeland and barren. Methods such as BSNet and S-RA-FCN achieve good results in the edge part, but in part with a relatively large gap within the class, such as the third row in Fig. 8, errors are prone to occur. However, our method not only has the same performance as BSNet in the edge part but also maintains high accuracy inside the object. The results are consistent with our previous analysis, showing the real effectiveness of our method.

C. Reason Analysis of Performance Improvement

The results show that the main bottleneck of performance improvement is the overlap of different categories in the feature space. This overlap is mainly determined by the similar appearance between different categories. The visual features extracted by various methods are shown in Fig. 9. It can be seen that the features extracted by the previous methods, such as PSPNet, U-Net, and Deeplabv3+, are confusing. Each category has no obvious boundaries, some areas are overemphasized, and some are completely unresponsive. INfoNCE loss-based Contrastseg [20] and attention-based HRNet + OCR achieve great improvement in performance, and correspondingly, the features extracted are also greatly improved in discrimination. Contrastseg responds very well to yellow agriculture but badly to the rangeland. This phenomenon is consistent with our previous analysis that the traditional contrastive learning methods squeeze the feature space of classes that are not in the current batch. And the rangeland category may be squeezed. The attention-based method such as HRNet + OCR [39] has obtained quite good results, which are consistent with Contrastseg. But compared with our method, the completeness is still a little worse. Our method guarantees the feature space of the agriculture category while ensuring the rangeland, so the regions of these two categories have good correspondence. In summary, the reason why our method achieves such good results is that we make different categories distinguish each other and do not invade each other.

D. Results With Different Base Networks

In the above results, the baseline used is Deeplabv3+ [34] with ResNet-50 as the backbone. To verify the generality of

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 02:44:19 UTC from IEEE Xplore. Restrictions apply.

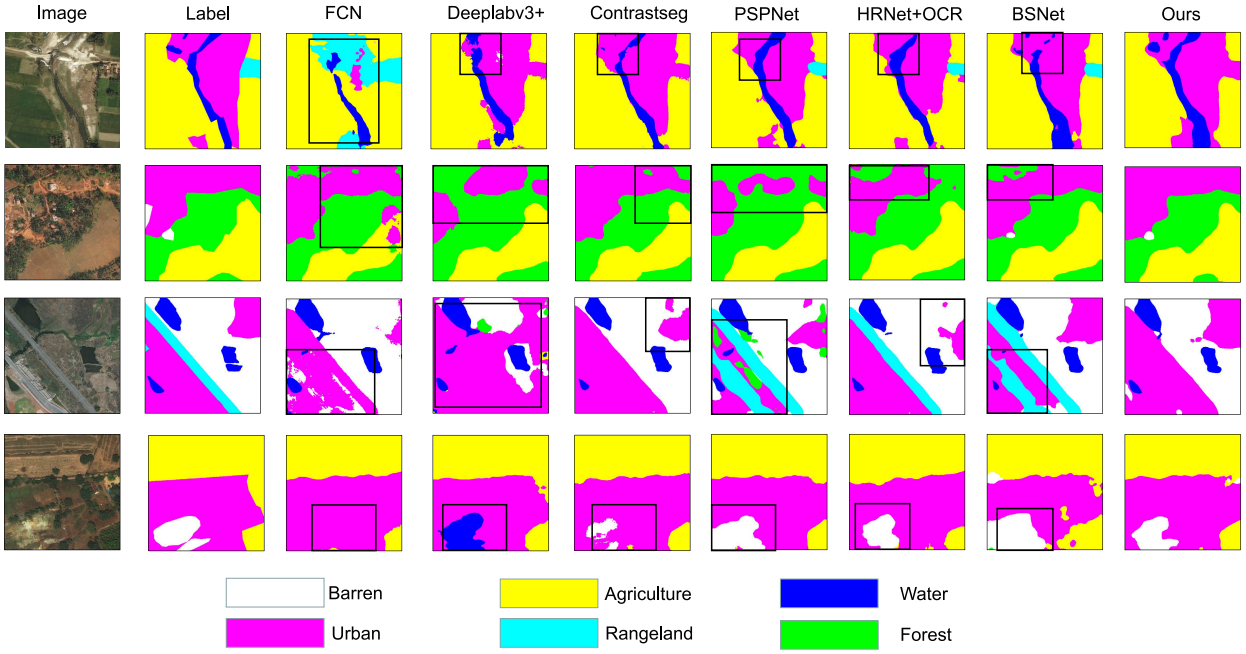


Fig. 8. Visualization results of different methods on the Deepglobe dataset. It can be seen that the area of improvement is wide and obvious.

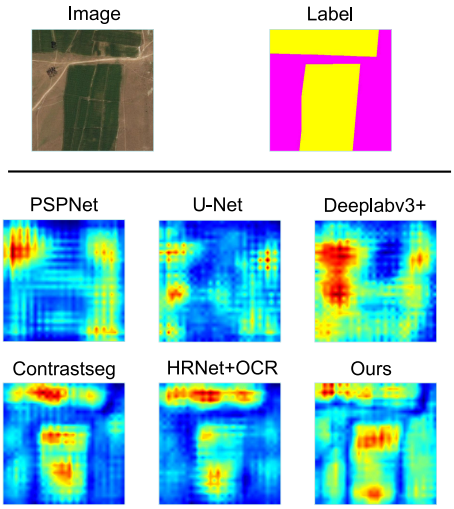


Fig. 9. Visualization of the extracted features for various methods. Above the black line is the image and the corresponding label, and below is the visualization of the features extracted by various methods.

our method, we take the other networks as the base network for the experiments. PSPNet [6] is a segmentation network based on a PPM, while DANet [9] is a segmentation model with an attention mechanism, and they are two basic structures of semantic segmentation. So it is very representative to take the two as the basic segmentation network. The results of PSPNet and DANet as basic segmentation networks are shown in Table III. It can be seen that our method has significant improvement on different segmentation networks, which fully demonstrates the universality of our method. It can also be seen that our performance is significantly greater than Contrastseg [20], which demonstrates the unique capabilities of our works for aerial imagery.

TABLE III
RESULTS ON DEEPGLOBE WITH DIFFERENT BASE NETWORKS

Base Network	Addition	mIoU (%)
PSPNet [6]	None	52.8
	Contrastseg [20]	54.9
	Ours	**57.1**
DANet [9]	None	58.7
	Contrastseg [20]	61.3
	Ours	**64.9**

E. Results on Potsdam and Vaihingen, Limitation Analysis, and Future Work

We conduct experiments on Potsdam and Vaihingen. The results are shown in Tables IV and V. The visualization results are shown in Fig. 10. Our method has obtained certain improvements from quantitative results and visual analysis. But compared with the impressive improvements on Deepglobe and iSAID, the results on Potsdam and Vaihingen are weak.

1) Limitation Analysis: We analyze the reason here. Potsdam and Vaihingen have relatively high intraclass aggregation compared with Deepglobe and iSAID, i.e., their background areas are few and not involved in testing, and there are no easily confusing categories. On the other hand, the data scale of Potsdam and Vaihingen is small, and our computation of prototypes, especially dataset-level prototypes, requires a large amount of data and many iterations. Most importantly, the difficulty with the Potsdam and Vaihingen datasets is that there

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 02:44:19 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
RESULTS ON POTSDAM

Method	Backbone	Impervious Surface	Building	Low Vegetation	Tree	Car	mean F_1	OA	mIoU
FCN-8s [48]	ResNet-50	89.9	93.7	83.0	85.2	93.5	89.1	87.8	71.7
U-Net [5]	ResNet-50	88.2	91.1	82.8	84.9	91.6	87.7	86.2	68.5
SegNet [38]	ResNet-50	87.8	90.7	81.0	84.7	89.7	86.8	85.7	66.2
EncNet [49]	ResNet-50	91.0	94.9	84.4	85.9	93.6	90.0	89.0	73.2
RefineNet [50]	ResNet-50	88.1	93.1	85.6	86.3	90.3	88.7	88.3	72.6
CCEM [51]	ResNet-50	88.3	93.2	84.7	86.0	92.8	89.0	89.1	72.8
S-RA-FCN [52]	ResNet-50	90.7	94.2	83.8	85.8	93.6	89.6	88.5	72.5
PSPNet [6]	ResNet-50	90.7	94.8	84.1	85.9	90.5	89.2	88.7	71.7
BSNet [47]	ResNet-50	92.4	**95.6**	86.8	88.1	**94.6**	91.5	90.7	77.5
DeepLabV3+ (baseline) [34]	ResNet-50	91.3	94.8	84.2	86.6	93.8	90.1	89.2	73.8
Ours	ResNet-50	**93.2**	95.0	**87.4**	**88.6**	94.1	**91.7**	**91.4**	**78.6**

TABLE V
RESULTS ON VAIHINGEN

Method	Backbone	Impervious Surface	Building	Low Vegetation	Tree	Car	mean F_1	OA	mIoU
FCN-8s [48]	ResNet-50	90.0	93.0	77.7	86.5	80.4	85.5	88.3	75.5
U-Net [5]	ResNet-50	90.5	93.3	79.6	87.5	76.4	85.5	89.2	75.5
SegNet [38]	ResNet-50	90.2	93.7	78.5	85.8	83.9	86.4	88.5	76.8
EncNet [49]	ResNet-50	91.2	94.1	79.2	86.9	83.7	87.0	89.4	77.8
RefineNet [50]	ResNet-50	91.1	94.1	79.8	87.2	82.3	86.9	88.9	77.1
CCEM [51]	ResNet-50	91.5	93.8	79.4	87.3	83.5	87.1	88.6	78.0
S-RA-FCN [52]	ResNet-50	90.5	93.8	79.6	87.5	82.6	86.8	89.2	77.3
PSPNet [6]	ResNet-50	90.6	94.3	79.0	87.0	70.7	84.3	89.1	74.1
BSNet [47]	ResNet-50	**92.1**	94.4	83.1	88.3	86.7	88.9	90.3	80.2
DeepLabV3+ (baseline) [34]	ResNet-50	91.4	94.7	79.6	87.6	85.8	87.8	89.9	79.0
Ours	ResNet-50	91.8	**95.2**	**83.3**	**88.6**	**87.4**	**89.3**	**90.6**	**80.7**

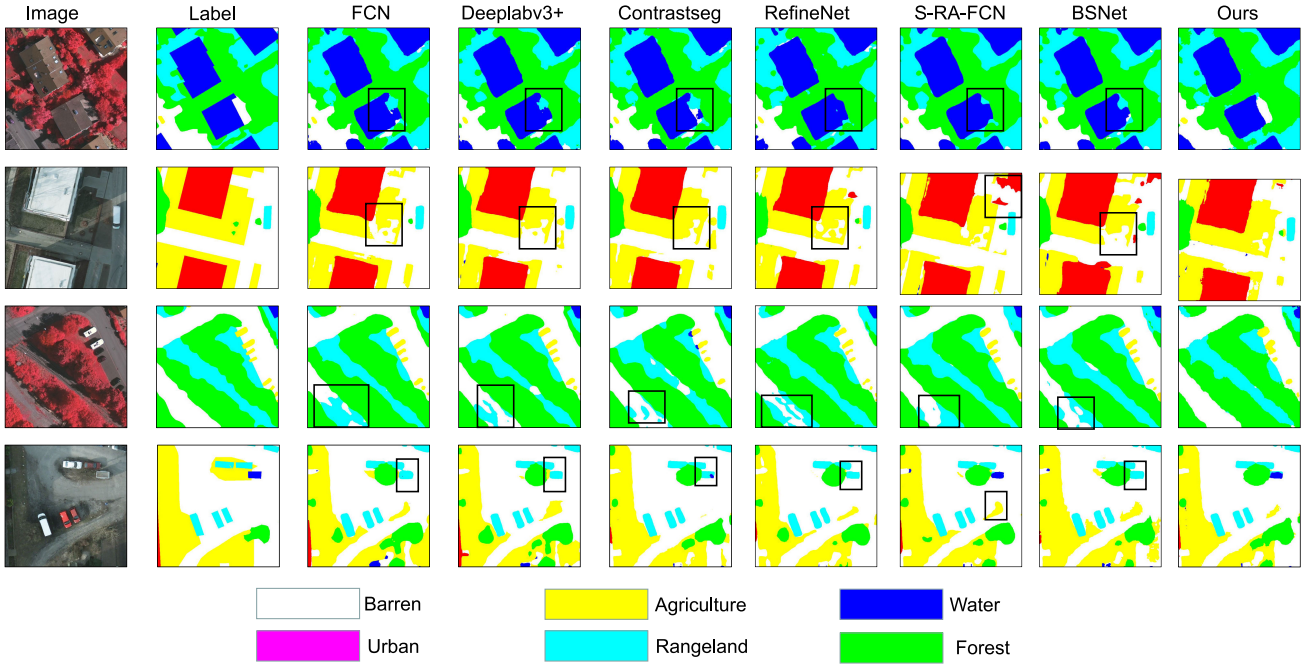


Fig. 10. Visualization results of different methods on the Potsdam and Vaihingen datasets. The first line is from the Vaihingen dataset, and the second is from the Potsdam dataset. Others are from the Deepglobe dataset.

are some objects from the same category that contains totally different appearance from others.

2) Future Work: For these problems, future studies show how to obtain representative prototypes for categories and perform more effective and efficient feature compaction and dispersion. Obviously, setting a prototype (although it is multilevel) for a class is very restrictive, so using multiple prototypes to represent different parts of a class is a more

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 02:44:19 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
RESULTS ON DEEPGLOBE WITH DIFFERENT PLUG-IN CONTRASTIVE LEARNING METHODS

Base Network	Contrastive learning methods	mIoU (%)
Deeplabv3+ [34]	[53]	61.0
	Contrastseg [20]	62.5
	Ours	**67.7**
DANet [9]	[53]	59.4
	Contrastseg [20]	61.3
	Ours	**64.9**

appropriate strategy. How to obtain multiple prototypes of a category so that each prototype is different from the other to represent all parts of the category is a possible direction in the future. Meanwhile, how to perform dispersion and comparison under multiple prototypes also needs to be redesigned, which can be obtained from self-learning and unsupervised learning.

F. Comparison With Plug-In Contrastive Learning Methods

We compare our methods with the plug-in contrastive learning methods [20], [53], and all the methods are in a unified baseline network, i.e., Deeplabv3+ with ResNet-50 as the backbone, and DANet with ResNet-50 as the backbone. All the hyperparameters are chosen to give the best results in the original article. The results of the experiment are shown in Table VI. The effect of [53] is the worst, which is easy to understand. [53] is designed for semi-supervised semantic segmentation and is targeted at natural images. Contrastseg is a fully supervised contrastive learning, but it aims at natural images and does not consider the characteristics of aerial images, so the effect is relatively poor. Our method is specifically for aerial images and achieves the best results.

In a nutshell, our method differs from Contrastseg [20] and [53] in three main points.

1) Contrastseg [20] and [53] are based on InfoNCE loss, which needs to sample negative samples, and it has high requirements for the selection of anchor features and features for comparison. However, our method decouples compaction and dispersion, so there is no high need to sample negative samples. In addition, we calculate a multilevel prototype, which is highly representative and very suitable as anchor features.
2) Contrastseg [20] and [53] are all pixel-level comparisons (pixel-pixel or pixel-local prototype), but ours is multilevel (pixel level and prototype level), and the prototypes are also divided into global and local types. This allows the feature distribution to be more fully optimized.
3) The dispersion and compaction of Contrastseg [20] and [53] are relative, i.e., the distance of features within a class is smaller than the distance of features between classes. On the contrary, we directly optimize the absolute distance so that the feature distribution of other classes will not be affected during the operation, and the dispersion within the class and the compaction between the classes will be more thorough.

TABLE VII
TRAINING TIME COMPARISON

Model	Module	Forward	Loss	Backward	All
Deeplabv3+	None	0.223s	0.002s	0.649s	0.874s
	Ours	0.251s	0.052s	0.815s	1.118s

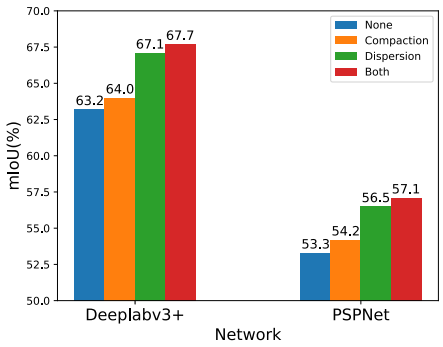


Fig. 11. Respective roles of compaction and dispersion. Although the performance gap between Deeplabv3+ and PSPNet is large, our approach brings significant improvements in both the networks.

G. Computational Complexity Analysis

Computational complexity analysis is important for evaluating the performance of an algorithm in practical application, so we show the cost during training and testing.

1) Test or Inference: Since our work does not change the structure of the network, i.e., a particular network with or without MFC and MFD is completely consistent during testing or inference. Therefore, the computational complexity during inference is completely consistent with the original network.

2) Training: The comparison of training time is shown in Table VII. The results are obtained when using two RTX 3090 GPUs for an iteration. The total training time is divided into forward time, loss time, and backward time. Since we use more losses (such as mse loss) and use a memory bank to store some features and prototypes, the loss time and backward time with our method increase. However, compared with the increase in training time, the increase in accuracy may be more meaningful for practical use.

H. Ablation Study

To verify the validity of the proposed modules and operations, as well as the influence of various hyperparameters on the final results, sufficient experiments are conducted. We conduct ablation studies on the dataset Deepglobe and use Deeplabv3+ with ResNet-50 as the base segmentation network.

1) Respective Roles of Compaction and Dispersion: We use mse loss so that the dispersion and compacting operations are separated. In this section, we show the respective roles of the two operations. To ensure the effectiveness of the experiment, we conduct verifications on both Deeplabv3+ and PSPNet. The experimental results are shown in Fig. 11, where "none" represents the baseline. Compaction represents performing only intraclass compaction, and dispersion indicates that only interclass dispersion is performed. As can be seen, any of these two actions can improve the results, and the combination of the two is the best. In comparison, the

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 02:44:19 UTC from IEEE Xplore. Restrictions apply.

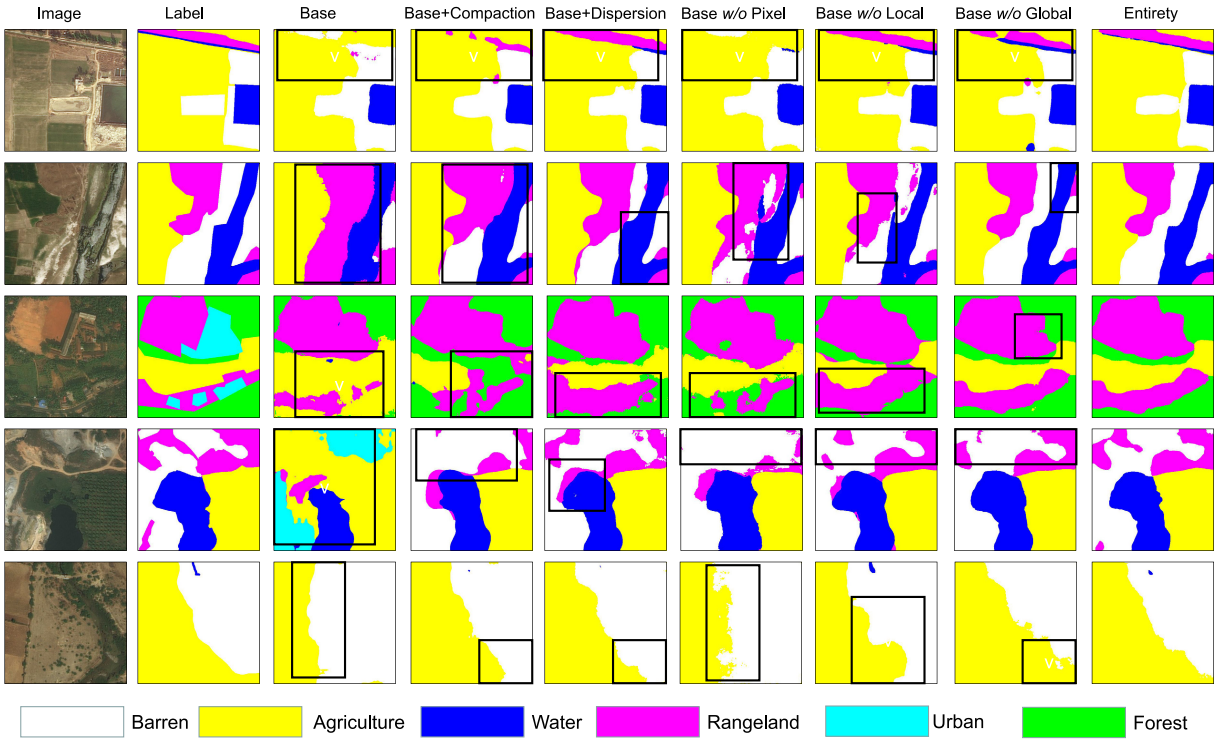


Fig. 12. Visualization of the output of the network under different settings. The base represents Deeplabv3+ with ResNet-50 as the backbone. Black boxes show areas where predictions are wrong. Base + Compaction means that the network only conducts compacting operations, and Base + Dispersion is that the network only conducts dispersing operations. *w/o* Pixel indicates that the network discards the pixel-level operations, and *w/o* local and *w/o* global are similar.

TABLE VIII
RESULTS WITH DIFFERENT LEVEL OPERATIONS

pixel-level	local-level	global-level	mIoU (%)
✓	✓	✓	67.7
	✓	✓	65.4
✓		✓	66.5
✓	✓		67.1
✓			65.8
	✓		64.2
		✓	63.4
			63.2

effect of dispersing operations between classes is slightly more obvious than that of compaction within classes. With intuitive thinking, the final result is obtained by comparing logits of other classes in the classifier head, so it is understandable that the effect of interclass dispersion is more obvious. As shown in columns 4 and 5 of Fig. 12, the two operations significantly improved the segmentation accuracy, and the effect of dispersion is more obvious.

2) Results With Different Level Operations: The dispersing and compacting operations are performed on the pixel, local, and global levels. In this section, we discuss the impact of different levels of operations. Note that except for different levels of operation, other settings remain the same. The experimental results are shown in Table VIII. Experiments with different level operations are carried out respectively. It can be seen from the results that the pixel-level operation has the most remarkable effect on the improvement, and only the pixel-level operation can obtain a great improvement. Moreover, pixel level can be considered as the basis of other operations. Only local-level or global-level results are not satisfied, but combined with pixel level, it can exert obvious advantages. This phenomenon is easily understandable, i.e., higher dimensional operations can take advantage only after the basic features satisfy a certain distribution. The local-level operation can improve by 1.0 points relative to the baseline. However, the combination with pixel level can improve by 1.3 points compared with the result of only pixel level. The local level can optimize the feature distribution again based on the original pixel level. The global-level operation alone has a very limited effect and can only bring about a 0.2-point improvement, but the cooperation with the pixel level and local level can bring about a 0.6-point improvement. In summary, all three levels of combined operations outperform any of them. Local and global prototypes can be thought of as higher level enhancements and additions to pixel-level operations. The visualization without different level operations is shown in columns 6–8 of Fig. 12.

3) Results With Different Memory Bank Sizes: Memory bank size determines the number of features, and some works [20] have demonstrated the importance of the number of selected contrastive features for final results. Therefore, we validate the effect of different memory bank sizes on Contrastseg and our method. Note that the memory bank reduction does not affect global prototypes but only pixel-level

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 02:44:19 UTC from IEEE Xplore. Restrictions apply.

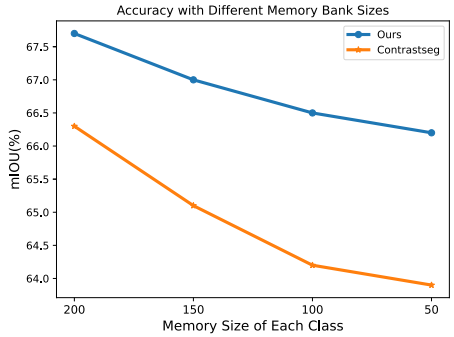


Fig. 13. Accuracy change with different memory bank sizes. Our method is less affected by changes in memory bank size.

TABLE IX
RESULTS ON DEEPGLOBE WITH AND WITHOUT CONSISTENT LOSS

Method	mIoU(%)
with consistency	67.7
without consistency	66.1 (-1.6)

TABLE X
RESULTS ON DEEPGLOBE WITH DIFFERENT COEFFICIENTS

λ_1	λ_2	mIoU(%)
10	10	67.7
50	50	66.2
10	50	67.2
50	10	66.4

features and local prototypes. The experimental results are shown in Fig. 13. It is obvious that with the decrease in the memory bank, the effect is gradually weakened. However, compared with Contrastseg, our method is less affected by the memory bank. This is understandable since Contastseg relies entirely on the memory bank to select contrastive features, but in our works, the local prototype can also select contrast features from the extracted features, and the global prototype-level operation is also not affected. In conclusion, multilevel interclass dispersion and intraclass compaction are more robust and require less memory bank.

4) Results Without Consistency of Feature Extractor and Classifier: In the method section, we analyze the significance of maintaining the consistency of the feature extractor and classifier, and here we demonstrate it through experiments. The results with and without consistent losses are shown in Table IX. It can be seen that if the consistent loss is not used, the result will be significantly decreased, so the consistent loss is indispensable.

5) Results With Different Coefficients: Since the number of pixel features is more than that of the local and global prototypes, we add a coefficient to the loss of the local and global prototype as shown in (9). In this section, we verify the influence of the coefficient on the experimental results. The result is shown in Table X. It can be seen that the coefficients have little influence on the overall performance, and the network performance is very good in the range of 10–50.

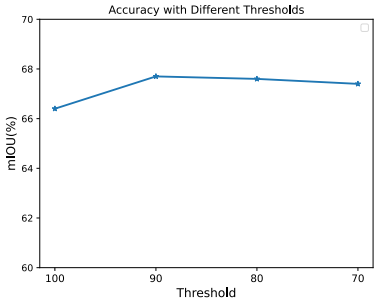


Fig. 14. Results with different thresholds of local prototypes.

TABLE XI
RESULTS WITH DIFFERENT VALUES OF V

Values of V	mIoU(%)
1024	67.7
512	67.6
256	67.3

The results are best when the coefficients of compaction (λ_1) and dispersion ($\lambda 2$) are 10. An increase in either of the two coefficients will lead to performance degradation, which may be related to the poor representation of the calculated prototypes.

6) Results With Different Thresholds of β: When calculating the local prototype, it is necessary to filter features with high confidence. To explore the influence of the confidence threshold on final results, we set different thresholds for experiments. The result is shown in Fig. 14. It can be seen that selecting the first 90% feature works best, followed by 80%, but both are significantly better than 100%. The experimental results show that there are outliers in the extracted features, but not many, so deleting a small number of outliers can achieve a good performance.

7) Results With Different Values of V: In this section, we verify the effect of different values of V on the final result. We use Deeplabv3+ with ResNet-50 as the backbone. The results of the experiments are shown in Table XI. It can be seen that when V takes 1024, the result is the same. And when taking 512 or 256, the result dropped slightly. But our method is relatively insensitive to V, and the effect changes little in the range of 512–1024.

V. CONCLUSION

In this article, we first analyze the differences between aerial images and natural images, and then, based on the difference, propose the multilevel interclass dispersion and intraclass compaction operations. This multilevel operation constructs the extracted features from all the aspects so that the features that are easy to confuse can be easily distinguished.

This is a different way from the previous design of the network structure, and it is an improved way from the level of feature compaction and dispersion. The previous land classification works of aerial images focus more on the attention mechanism, and we think conducting some operations at the feature level is also the correct direction. The semantic

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 02:44:19 UTC from IEEE Xplore. Restrictions apply.

segmentation of aerial images is different from that of natural images, and it has many unique characteristics. For example, an aerial image has fewer categories, the objects in the aerial image have no fixed shape, and the edges are staggered vertically and horizontally. In view of these characteristics, how to calculate a more representative prototype and how to better select anchor features and contrastive features for aerial images are future explorations based on this work. At the same time, how to effectively combine attention mechanisms and feature-level operations is also a valuable research topic.

REFERENCES

[1] Z. Lv, F. Wang, G. Cui, J. A. Benediktsson, T. Lei, and W. Sun, "Spatial–spectral attention network guided with change magnitude image for land cover change detection using remote sensing images," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 4412712.

[2] Z. Lv, F. Wang, W. Sun, Z. You, N. Falco, and J. A. Benediktsson, "Landslide inventory mapping on VHR images via adaptive region shape similarity," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5630211.

[3] Z. Lv, H. Huang, L. Gao, J. A. Benediktsson, M. Zhao, and C. Shi, "Simple multiscale UNet for change detection with heterogeneous remote sensing images," *IEEE Geosci. Remote Sens. Lett.*, vol. 19, pp. 1–5, 2022.

[4] Z. Lv, G. Li, Z. Jin, J. A. Benediktsson, and G. M. Foody, "Iterative training sample expansion to increase and balance the accuracy of land classification from VHR imagery," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 1, pp. 139–150, Jan. 2021.

[5] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in *Proc. Int. Conf. Med. Image Comput. Comput.-Assist. Intervent.* Cham, Switzerland: Springer, 2015, pp. 234–241.

[6] H. Zhao, J. Shi, X. Qi, X. Wang, and J. Jia, "Pyramid scene parsing network," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 6230–6239.

[7] L.-C. Chen, G. Papandreou, F. Schroff, and H. Adam, "Rethinking Atrous convolution for semantic image segmentation," 2017, *arXiv:1706.05587*.

[8] J. Hu, L. Shen, and G. Sun, "Squeeze-and-excitation networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 7132–7141.

[9] J. Fu et al., "Dual attention network for scene segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 3141–3149.

[10] H. Li, P. Xiong, J. An, and L. Wang, "Pyramid attention network for semantic segmentation," 2018, *arXiv:1805.10180*.

[11] G. Deng, Z. Wu, C. Wang, M. Xu, and Y. Zhong, "CCANet: Class-constraint coarse-to-fine attentional deep network for subdecimeter aerial image semantic segmentation," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 4401120.

[12] W. Chen, Z. Jiang, Z. Wang, K. Cui, and X. Qian, "Collaborative global-local networks for memory-efficient segmentation of ultra-high resolution images," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 8916–8925.

[13] L. Ding, J. Zhang, and L. Bruzzone, "Semantic segmentation of large-size VHR remote sensing images using a two-stage multiscale training architecture," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 8, pp. 5367–5376, Aug. 2020.

[14] L. Shan and W. Wang, "DenseNet-based land cover classification network with deep fusion," *IEEE Geosci. Remote Sens. Lett.*, vol. 19, pp. 1–5, 2022.

[15] Q. Ming, L. Miao, Z. Zhou, and Y. Dong, "CFC-Net: A critical feature capturing network for arbitrary-oriented object detection in remote-sensing images," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5605814.

[16] Q. Liu, M. Kampffmeyer, R. Jenssen, and A.-B. Salberg, "Dense dilated convolutions' merging network for land cover classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 9, pp. 6309–6320, Sep. 2020.

[17] S. W. Zamir et al., "iSAID: A large-scale dataset for instance segmentation in aerial images," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops*, Jul. 2019, pp. 28–37.

[18] I. Demir et al., "DeepGlobe 2018: A challenge to parse the Earth through satellite images," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, Jun. 2018, pp. 172–181.

[19] A. van den Oord, Y. Li, and O. Vinyals, "Representation learning with contrastive predictive coding," 2018, *arXiv:1807.03748*.

[20] W. Wang, T. Zhou, F. Yu, J. Dai, E. Konukoglu, and L. V. Gool, "Exploring cross-image pixel contrast for semantic segmentation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 7283–7293.

[21] R. Achanta, A. Shaji, K. Smith, A. Lucchi, P. Fua, and S. Süsstrunk, "SLIC superpixels compared to state-of-the-art superpixel methods," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 34, no. 11, pp. 2274–2282, Nov. 2012.

[22] T.-W. Chen, Y.-L. Chen, and S.-Y. Chien, "Fast image segmentation based on K-means clustering with histograms in HSV color space," in *Proc. IEEE 10th Workshop Multimedia Signal Process.*, Oct. 2008, pp. 322–325.

[23] M. Golipour, H. Ghassemian, and F. Mirzapour, "Integrating hierarchical segmentation maps with MRF prior for classification of hyperspectral images in a Bayesian framework," *IEEE Trans. Geosci. Remote Sens.*, vol. 54, no. 2, pp. 805–816, Feb. 2016.

[24] K. Sun et al., "High-resolution representations for labeling pixels and regions," 2019, *arXiv:1904.04514*.

[25] H. Zhang et al., "ResNeSt: Split-attention networks," 2020, *arXiv:2004.08955*.

[26] L. Ding, H. Tang, and L. Bruzzone, "LANet: Local attention embedding to improve the semantic segmentation of remote sensing images," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 1, pp. 426–435, Jan. 2021.

[27] K. Heidler, L. Mou, C. Baumhoer, A. Dietz, and X. X. Zhu, "HED-UNet: Combined segmentation and edge detection for monitoring the Antarctic coastline," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 4300514.

[28] R. Hadsell, S. Chopra, and Y. LeCun, "Dimensionality reduction by learning an invariant mapping," in *Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit.*, Jun. 2006, pp. 1735–1742.

[29] Z. Wu, Y. Xiong, S. X. Yu, and D. Lin, "Unsupervised feature learning via non-parametric instance discrimination," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 3733–3742.

[30] M. Ye, X. Zhang, P. C. Yuen, and S.-F. Chang, "Unsupervised embedding learning via invariant and spreading instance feature," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 6203–6212.

[31] Y. Chen, Y. Cao, H. Hu, and L. Wang, "Memory enhanced global-local aggregation for video object detection," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 10334–10343.

[32] S. Li, D. Chen, B. Liu, N. Yu, and R. Zhao, "Memory-based neighbourhood embedding for visual recognition," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 6101–6110.

[33] X. Wang, H. Zhang, W. Huang, and M. R. Scott, "Cross-batch memory for embedding learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 6387–6396.

[34] L.-C. Chen, Y. Zhu, G. Papandreou, F. Schroff, and H. Adam, "Encoder–decoder with Atrous separable convolution for semantic image segmentation," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 801–818.

[35] H.-M. Yang, X.-Y. Zhang, F. Yin, and C.-L. Liu, "Robust classification with convolutional prototype learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 3474–3482.

[36] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Jun. 2009, pp. 248–255.

[37] H. Zhao, X. Qi, X. Shen, J. Shi, and J. Jia, "ICNet for real-time semantic segmentation on high-resolution images," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 405–420.

[38] V. Badrinarayanan, A. Kendall, and R. Cipolla, "SegNet: A deep convolutional encoder–decoder architecture for image segmentation," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 39, no. 12, pp. 2481–2495, Dec. 2017.

[39] Y. Yuan, X. Chen, X. Chen, and J. Wang, "Segmentation transformer: Object-contextual representations for semantic segmentation," 2019, *arXiv:1909.11065*.

[40] X. Li, Z. Zhong, J. Wu, Y. Yang, Z. Lin, and H. Liu, "Expectation-maximization attention networks for semantic segmentation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 9166–9175.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 02:44:19 UTC from IEEE Xplore. Restrictions apply.

[41] M. Yang, K. Yu, C. Zhang, Z. Li, and K. Yang, "DenseASPP for semantic segmentation in street scenes," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 3684–3692.
[42] X. Wang, R. Girshick, A. Gupta, and K. He, "Non-local neural networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 7794–7803.
[43] Z. Huang, X. Wang, L. Huang, C. Huang, Y. Wei, and W. Liu, "CCNet: Criss-cross attention for semantic segmentation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 603–612.
[44] A. Kirillov, R. Girshick, K. He, and P. Dollár, "Panoptic feature pyramid networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 6392–6401.
[45] S. Zheng et al., "Rethinking semantic segmentation from a sequence-to-sequence perspective with transformers," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 6877–6886.
[46] T. Xiao, Y. Liu, B. Zhou, Y. Jiang, and J. Sun, "Unified perceptual parsing for scene understanding," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2018, pp. 418–434.
[47] J. Hou, Z. Guo, Y. Wu, W. Diao, and T. Xu, "BSNet: Dynamic hybrid gradient convolution based boundary-sensitive network for remote sensing image segmentation," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5624022.
[48] J. Long, E. Shelhamer, and T. Darrell, "Fully convolutional networks for semantic segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2015, pp. 3431–3440.
[49] H. Zhang et al., "Context encoding for semantic segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 7151–7160.
[50] G. Lin, A. Milan, C. Shen, and I. Reid, "RefineNet: Multi-path refinement networks for high-resolution semantic segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 5168–5177.
[51] D. Marmanis, K. Schindler, J. D. Wegner, S. Galliani, M. Datcu, and U. Stilla, "Classification with an edge: Improving semantic image segmentation with boundary detection," *ISPRS J. Photogramm. Remote Sens.*, vol. 135, pp. 158–172, Jan. 2018.
[52] L. Mou, Y. Hua, and X. X. Zhu, "A relation-augmented fully convolutional network for semantic segmentation in aerial scenes," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 12408–12417.
[53] S. Liu, S. Zhi, E. Johns, and A. J. Davison, "Bootstrapping semantic segmentation with regional contrast," 2021, *arXiv:2104.04465*.



Lianlei Shan (Graduate Student Member, IEEE) received the B.E. degree from the Shandong University of Science and Technology, Qingdao, China, in 2018. He is currently pursuing the Ph.D. degree with the School of Computer and Control Engineering, University of Chinese Academy of Sciences, Beijing, China.

His research interests include remote sensing image processing and computer vision.



Weiqiang Wang (Member, IEEE) received the B.E. and M.E. degrees in computer science from Harbin Engineering University, Harbin, China, in 1995 and 1998, respectively, and the Ph.D. degree in computer science from the Institute of Computing Technology, Chinese Academy of Sciences (CAS), Beijing, China, in 2001.

He is currently a Professor with the School of Computer and Controlling Engineering, University of CAS, Beijing. His research interests include multimedia content analysis and computer vision.



Ke Lv (Senior Member, IEEE) received the B.S. degree from the Department of Mathematics, Ningxia University, Yinchuan, China, in 1993, and the master's and Ph.D. degrees from the Department of Mathematics and the Department of Computer Science, Northwest University, Xi'an, China, in 1998 and 2003, respectively.

He is currently a Professor with the University of Chinese Academy of Sciences, Beijing, China. His research focuses on curve matching, 3-D image construction, and computer graphics.



Bin Luo (Senior Member, IEEE) received the B.Eng. degree in electronics and the M.Eng. degree in computer science from Anhui university, Hefei, China, in 1984 and 1991, respectively, and the Ph.D. degree in computer science from the University of York, York, U.K., in 2002.

He joined the University of York as a Research Associate, from 2000 to 2004.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 02:44:19 UTC from IEEE Xplore. Restrictions apply.